

第18回 処理をメソッドに分ける

今回のゴール

- 処理のまとまりをメソッドにして、`main` から呼び出せる。
- 引数と戻り値を使ったメソッドを自分で書ける。
- 戻り値を受け取らないと値が変わらない理由を説明できる。

前回とのつながり

第17回で、コマンドを選べるバトルができた。しかし `main` の中にすべてが書かれていて長く、どこで何をしているか探しにくい。今回は、同じ形のコードや意味のまとまりを **メソッド** に分ける。バトルの動きは第17回と同じまま、コードを読みやすくする。

実習時のフォルダ構成

```
ワークスペース
├─ java18_sample
│   └─ src
│       └─ Main.java
└─ java18_exercise
    └─ src
        └─ Main.java
```

- 配布ファイル: `プログラム/第18回/sample/Main.java`、`プログラム/第18回/exercise/Main.java`
 - 解答例: `プログラム/第18回/answer/Main.java`
-

講義内容

1. メソッドとは

メソッドは、処理に名前を付けてまとめたもの。`main` もメソッドの 1 つ。

```
// 定義（作る）
static int heal(int hp, int maxHp, int amount) {
    hp += amount;
    if (hp > maxHp) {
        hp = maxHp;
    }
    return hp;
}

// 呼び出し（使う）
heroHp = heal(heroHp, heroMaxHp, 15);
```

部分	例	意味
戻り値の型	<code>int</code>	返す値の型。何も返さないときは <code>void</code>
メソッド名	<code>heal</code>	何をするか分かる名前にする
引数	<code>(int hp, int maxHp, int amount)</code>	呼び出すときに渡す値を受け取る変数
<code>return</code>	<code>return hp;</code>	値を呼び出し元へ返して、メソッドを終わる

- 今は、`main` から呼び出すメソッドには `static` を付けると覚える（理由は第25回で学ぶ）。
- メソッドは `main` の `}` の後、クラスの `}` の前に書く。

2. 戻り値を受け取る

```
heal(heroHp, heroMaxHp, 15);           // これだけでは heroHp は変わらない
heroHp = heal(heroHp, heroMaxHp, 15);  // 戻り値を代入して、はじめて変わる
```

- 引数の `hp` は、`heroHp` の **値のコピー** を受け取った別の変数。メソッドの中で `hp` を変えても、`heroHp` は変わらない。
- だから、結果を `return` で返し、呼び出し側で代入する。

3. 引数の数・型が違う同じ名前のメソッド（オーバーロード）

```
static void showStatus(String name, int hp, int maxHp) { ... } // 勇者用
static void showStatus(String name, int hp) { ... }           // 敵用
```

- 引数の数や型が違えば、同じ名前のメソッドを複数作れる。これを **オーバーロード** という。
- 呼び出したときの引数に合うものが選ばれる。自分で作れなくてよいが、読んで「どちらが呼ばれるか」分かるようにする。

- `main(String[] args)` の `args` も引数。起動するときに文字列を渡すための引数だが、この授業では使わない。

4. 小数を整数にする(キャスト)

会心の一撃を 1.5 倍に変えた。

```
return (int) (attack * 1.5);
```

- `attack * 1.5` は小数(`double` 型)になる。`8 * 1.5` は `12.0`。
- `int` 型で返すために、`(int)` を付けて整数に変換する。これを **キャスト** という。小数点以下は切り捨てられる。
- `(int)` を付けないと、「double から int への変換」のコンパイルエラーになる。

5. サンプルを読む

プログラム/第18回/sample/Main.java (`main` の中は第17回とほぼ同じ。変わったところを中心に読む)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String[] commands = { "攻撃", "薬草を使う", "にげる" };

        // 勇者のステータス
        String heroName = "勇者";
        int heroHp = 30;
        int heroMaxHp = 30;
        int heroAttack = 8;
        int herbCount = 2;

        // 敵のステータス
        String enemyName = "ゴブリン";
        int enemyHp = 40;
        int enemyAttack = 7;

        int turn = 1;
        boolean escaped = false;

        System.out.println(enemyName + "があらわれた!");

        while (heroHp > 0 && enemyHp > 0) {
            System.out.println();
            System.out.println("--- ターン" + turn + " ---");
            showStatus(heroName, heroHp, heroMaxHp);
            showStatus(enemyName, enemyHp);

            int command = inputCommand(scanner, commands);

            switch (command) {
                case 1:
```

```

        int damage = calcDamage(heroAttack, turn);
        enemyHp -= damage;
        System.out.println(heroName + "の攻撃！ " + enemyName + "に" + damage + "のダメージ");
        break;
    case 2:
        if (herbCount > 0) {
            heroHp = heal(heroHp, heroMaxHp, 15);
            herbCount--;
            System.out.println(heroName + "は薬草を使った！ HPが" + heroHp + "になった");
        } else {
            System.out.println("薬草がない！");
        }
        break;
    case 3:
        System.out.println(heroName + "はにげだした！");
        escaped = true;
        break;
}

if (escaped || enemyHp <= 0) {
    break;
}

heroHp -= enemyAttack;
System.out.println(enemyName + "の攻撃！ " + heroName + "に" + enemyAttack + "のダメージ");
turn++;
}

System.out.println();
if (escaped) {
    System.out.println("うまくにげされた。");
} else if (enemyHp <= 0) {
    System.out.println(enemyName + "をたおした！");
} else {
    System.out.println(heroName + "はたおれてしまった...");
}
}

// 名前と HP を表示する（最大 HP あり）
static void showStatus(String name, int hp, int maxHp) {
    System.out.println(name + " HP:" + hp + "/" + maxHp);
}

// 名前と HP を表示する（最大 HP なし）
static void showStatus(String name, int hp) {
    System.out.println(name + " HP:" + hp);
}

// コマンドの一覧を表示し、正しい番号が入力されるまで聞いて、その番号を返す
static int inputCommand(Scanner scanner, String[] commands) {
    for (int i = 0; i < commands.length; i++) {
        System.out.println((i + 1) + ": " + commands[i]);
    }
    int command;
    do {

```

```

        System.out.print("コマンドを選ぶ > ");
        command = scanner.nextInt();
    } while (command < 1 || command > commands.length);
    return command;
}

// 攻撃力とターン数から、ダメージを計算して返す (3 ターンごとに 1.5 倍)
static int calcDamage(int attack, int turn) {
    if (turn % 3 == 0) {
        System.out.println("会心の一撃!");
        return (int) (attack * 1.5);
    }
    return attack;
}

// 回復した後の HP を返す (最大 HP を超えない)
static int heal(int hp, int maxHp, int amount) {
    hp += amount;
    if (hp > maxHp) {
        hp = maxHp;
    }
    return hp;
}
}

```

- `main` が「何をしているか」だけの流れになり、細かい「どうやるか」はメソッドの中に移った。
- `inputCommand` は、配列 `commands` と `scanner` を引数で受け取っている。配列も引数にできる。

6. 実行する前に予想する

1、1、1、2、1、1 の順に入力する。

- 3 ターン目の会心の一撃のダメージはいくつか (`8 * 1.5` を `int` にする)。
- 何ターン目に勝つか。

7. 実行結果

ゴブリンがあらわれた！

--- ターン1 ---

勇者 HP:30/30

ゴブリン HP:40

1: 攻撃

2: 薬草を使う

3: にげる

コマンドを選ぶ > 1

勇者の攻撃！ ゴブリンに8のダメージ

ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン2 ---

勇者 HP:23/30

ゴブリン HP:32

1: 攻撃

```
2: 薬草を使う
3: にげる
コマンドを選ぶ > 1
勇者の攻撃！ ゴブリンに8のダメージ
ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン3 ---
勇者 HP:16/30
ゴブリン HP:24
1: 攻撃
2: 薬草を使う
3: にげる
コマンドを選ぶ > 1
会心の一撃！
勇者の攻撃！ ゴブリンに12のダメージ
ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン4 ---
勇者 HP:9/30
ゴブリン HP:12
1: 攻撃
2: 薬草を使う
3: にげる
コマンドを選ぶ > 2
勇者は薬草を使った！ HPが24になった
ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン5 ---
勇者 HP:17/30
ゴブリン HP:12
1: 攻撃
2: 薬草を使う
3: にげる
コマンドを選ぶ > 1
勇者の攻撃！ ゴブリンに8のダメージ
ゴブリンの攻撃！ 勇者に7のダメージ

--- ターン6 ---
勇者 HP:10/30
ゴブリン HP:4
1: 攻撃
2: 薬草を使う
3: にげる
コマンドを選ぶ > 1
会心の一撃！
勇者の攻撃！ ゴブリンに12のダメージ

ゴブリンをたおした！
```

8. エラーを体験する

1. `heroHp = heal(heroHp, heroMaxHp, 15);` の引数を 1 つ消して、`heal(heroHp, heroMaxHp)` にする。
2. 保存すると、引数が合わないというエラーになる。メッセージの中に、メソッドが受け取る引数の型 (`int, int, int`) が書かれていることを確認する。

3. 元に戻す。次に、`heroHp = heal(...)` の `heroHp =` を消して実行し、薬草を使っても HP が変わらないことを確かめる。これはエラーにならないが、結果がおかしい。確認したら元に戻す。
- コンパイルエラーは Pleiades が教えてくれるが、2 つ目のような「動くけれど結果が違う」間違いは、実行結果を見て自分で気付く必要がある。

演習1 サンプルを実行する

`java18_sample` でサンプルを実行し、予想と結果を比べる。

演習2 メソッドを完成させる

`java18_exercise` を作り、`プログラム/第18回/exercise/Main.java` を貼り付ける。

演習2-1 heal メソッドの中身を書く

- 今の `heal` は、受け取った `hp` をそのまま返すだけになっている。まず実行して、薬草を使っても HP が増えないことを確認する。
- `hp` に `amount` を足し、`maxHp` を超えたら `maxHp` にしてから返すように書く。

演習2-2 isAlive メソッドを作る

- HP を受け取り、0 より大きければ `true`、そうでなければ `false` を返す `isAlive` メソッドを作る。
- `while` の条件を、`isAlive` を使った形に書きかえる。

ヒント:

- 戻り値の型は `boolean`。`return hp > 0;` の 1 行で書ける。
- `while (isAlive(heroHp) && isAlive(enemyHp))`

演習2-3 HP バーを表示する

- HP 5 につき `■` を 1 個表示する `showHpBar(int hp)` メソッドを作る(戻り値なし)。
- 最大 HP ありの `showStatus` の中から呼び出し、勇者の HP の下にバーを表示する。

ヒント: `for (int i = 0; i < hp / 5; i++)` の中で `System.out.print("■");`。最後に `System.out.println();` で改行する。

完成条件(1 ターン目の表示):

```
--- ターン1 ---
勇者 HP:30/30
■■■■■
ゴブリン HP:40
```

薬草を使ったときに、HP が 30 を超えないことも確認する。

発展(任意)

- 敵の攻撃の処理を `enemyTurn` のようなメソッドにできるか考える。何を引数にして、何を戻り値にすればよいか。
- 敵用の `showStatus(String name, int hp)` にも HP バーを付ける。

確認問題

問1 出力は何か。

```
public static void main(String[] args) {
    int power = 10;
    int result = addBonus(power);
    System.out.println(power + " " + result);
}

static int addBonus(int attack) {
    attack += 5;
    return attack;
}
```

問2 出力は何か。

```
System.out.println((int) (9 * 1.5));
```

問3 出力は何か(2 行)。

```
public static void main(String[] args) {
    show("勇者");
    show(10);
}

static void show(int hp) {
    System.out.println("A");
}

static void show(String name) {
    System.out.println("B");
}
```

■ 答え

- 問1: `10 15` (`attack` は `power` の値のコピー。`power` は変わらない)
- 問2: `13` (`13.5` の小数点以下が切り捨てられる)
- 問3: `B` と `A` (引数の型に合うメソッドが選ばれる)

振り返り

- 戻り値の型・引数・`return` の役割を説明できるか。
- `heal(...)` の戻り値を代入しないと HP が変わらない理由を説明できるか。
- メソッドに分ける前と後で、どちらが読みやすいか。

次回へのつながり

勇者の情報は `heroName`、`heroHp`、`heroMaxHp`、`heroAttack` とばらばらの変数になっている。敵を 2 体、3 体と増やすと、変数がどんどん増えてしまう。今回は、キャラクターの情報を 1 つの **クラス** にまとめる。